

Angel Aldaco

661-334-0047 | aaaldaco2002@berkeley.edu | www.linkedin.com/in/angel-aldaco/ | [Personal Website](#)

TECHNICAL SKILLS

Languages: Java, Python, C/C++, SQL, JavaScript, HTML/CSS, Go, Verilog, R, RISC-V, Chisel

Developer Tools: Git, Docker, VS Code, Visual Studio, ROS, IntelliJ, MongoDB

Libraries: pandas, NumPy, Matplotlib

EDUCATION

University of California, Berkeley

Berkeley, CA

Bachelor of Arts in Computer Science, Political Science, Minor in Journalism

2025

- Relevant Coursework: Structure and Interpretations of Computer Programs, Data Structures, Designing Information Devices and Systems, Discrete Mathematics and Probability Theory, Effective Algorithms and Intractable Problems, Foundations of Data Science, Great Ideas of Computer Architecture (Machine Structures), Web Development, User Interface Design and Development, Operating Systems and Systems Programming, Digital Design and Integrated Circuits, Computer Architecture and Engineering, Computer Security, Database Systems, Introduction to Robotics, Designing STAR Assessments Tools

EXPERIENCE

Undergraduate Student Instructor

May 2022 – Present

University of California Berkeley

Berkeley, California

Computer Science Head Teaching Assistant, TA, Tutor

May 2022 - Aug 2024

- Head TA of Computer Security(CS161), Lead TA of Data Structures(CS61B), and Structures and Interpretations of Computer Programs(CS61B).
- Developed, reviewed, and did design reviews coding projects on data structures, memory management, Interpreters, security vulnerabilities, and cryptographic security, ensuring best practices in Java, Python, C, and Go. Projects Include: End-To-End File Sharing System Gitlet, NGordNet, Scheme Interpreter. (see projects Section)
- Gave Course wide Lectures on the Network Security Unit, Co-authored course materials and contributed to exam design, including questions on graph algorithms, and System Security.
- Mentored academic interns in pedagogy and technical concepts, helping to scale the instructional team for a high-enrollment course.
- Led discussion sections, labs, and office hours for 3+ computer science courses (CS161 - Computer Security, CS61B - Data Structures and Algorithms, CS61A - Structure and Interpretation of Computer Programs), collectively serving in 1,600+ students Courses.

PROJECTS

Pintos | *C, System Design*

August 2022 – December 2022

- Designed, Programmed, and Implemented an Operating System with a team of 4, Pintos including Handling User Programs, Argument Passing, System Calls, Strict Priority Scheduling for Threads, Dealing with User Threads, Buffer Caches, and Sub Directories
- Within Pintos Implemented Lists Structures for Kernels, a Functioning Shell with Operations Such As Pipelining, HTTP, and Implementing Memory Management

End to End Encrypted File System | *Go, System Design, Computer Security*

July 2023 – August 2023

- Designed and Implemented an Encrypted File System Where User Could: (1)Create Users, (2) Login with Correct Information, (3) Create Files, (4) Append to the File at a Constant Time, (5) Share Files to Other Users to Append and Read Files, (6) Revoke Access To File That was Previously Shared
- This Defended Against 2 Adversaries, One which Could See All of the Contents of the Database, And Another Which was a Revoked User Trying to Re-Gain Access to the File And Created Tests for such Adversaries

RookieDB | *Java, Database Management, SQL, MongoDB, NoSQL*

August 2023 – December 2023

- Implemented a DataBase System Including Implementing B+ Tree Indices, Join Algorithms, Query Optimizations, Concurrency with Queuing and Multigranularity Locks, and Recovery with the ARIES Recovery Algorithm. To Prepared For the Project, SQL Queries were coded up, and After the Project NoSQL entries (Using MongoDB) were Coded up.

- Parses Data CSVs on the History Of Words to Build an Interactive Data Base of That History in order to Build a Graph of Hyponyms and Hypernyms of Words, along with an Efficient Algorithm for Obtaining The Greatest Common Ancestor Between Words

Prairie Learn Element | *Python, JavaScript, HTML, CSS, Mustache, Design, Docker* January 2024 – May 2024

- Using PraireLearn's Pre-existing Code Base, Designing a New Element To More Efficiently Test Students Understanding of Sorting Algorithm Iteration per Iteration Through a Drag and Drop Mechanism with Randomized Elements in the List Along with Running a Study for it's Educational Use

Gitlet | *Java, Data Structures, Algorithms*

May 2023 - June 2023

- A Miniaturized Local Version of Git Which Mimics Git Behavior for init, add, commit, rm, log, global-log, find, status, restore, branch, switch, rm-branch, reset, and merge. This Requires A Clean System Design (Done Through an Updated Design Document) Which Implements Persistence Along with Maintaining Contents, Along with Implementing Graph Algorithms.

NumC | *C, Program Optimization*

April 2022 - May 2022

- A Version of NumPy Written for C with Matrix Operations implemented for C. These are Spedup with SIMD instructions, OpenMP, Algorithmic Optimizations (such as Repeated Squaring), Reducing Function Calls, Loop Unrolling, and Cache Blocking

Visual Servoing | *Python, NumPy, ROS*

January 2024 – February 2024

- Analyzing the Theory of Jointspace PD Velocity Control, Jointspace PD Torque Control, and Workspace Velocity Control
- Put a Jointspace PD Velocity Controller into Practice by Testing and Tuning the Gains of the Controller. As well as Testing it for a Linear Trajectory, a Circular Trajectory, and a Polygonal Trajectory Around a Tag.

Nonholonomic Control | *Python, NumPy, ROS, Controls, Applied Math, Path Planning* February 2024 - March 2024

- Explained and Analyzed Path Planning Methods on a Bicycle Model. Algorithms Include: Sinusoidal Planning, Rapidly-Exploring Random Trees Planning, and Optimization Based Planning.
- Following Implementation, This was Then Tested on an Obstacle Courses for All Planners Demonstrated Different Case-Scenarios One Would Use One Planner Over Another.

State Estimation | *Python, NumPy, ROS, Applied Math*

March 2024

- Analyzed and Implemented Classical State Estimation Models Including: Dead Reckoning, Kalman Filtering, and Extended Kalman Filtering.
- Following Implementation of Them, They Were Then Tested on Unicycle Model and a Quadcopter Model.

Grasp Planning with Sawyers | *Python, NumPy, ROS, Applied Math*

April 2024

- Analyzed and Implemented Classical State Estimation Models Including: Dead Reckoning, Kalman Filtering, and Extended Kalman Filtering.
- Following Implementation of Them, They Were Then Tested on Unicycle Model and a Quadcopter Model.

Discretized Model of a Soft Gripper | *Python, NumPy, ROS, Applied Math, Soft Robotics* April 2024 - May 2024

- Modeled and Implemented a Discretized Model of a Soft Gripper as Infinite Links on an Open Chain Manipulator. This is Often Done Using Bezier Curves Which We Did Model, But The Novel Aspect of The Project is Modeling It With Hermite Curves and Limit Analysis.
- Following Theory Analysis, and Implementation The Code Was Tested On Soft Grippers to See How Practical It Was In Comparison to Our Model.